

The EFGP-SING E-step

GP regression on uncertain latent velocities, solved by Fourier features

Abstract

This note specifies the E-step of SING with a GP drift prior, accelerated by EFGP. §2 states the whole contract: SING’s natural-gradient backbone asks a drift model for exactly three per-transition moments and `jax.grads` through them, so EFGP is fully described by three values plus six VJP slots, together with the three departures (D1–D3) from the generic SING algorithm. The rest is the *why*: the $q(f)$ update is GP regression on latent velocities (§4), made matrix-free by Stein’s lemma plus a Fourier basis whose Gram is BTTB Toeplitz (§5); the remaining $q(x)$ -expectations are Gaussian-enveloped NUFFT, computed by one of three interchangeable backends (§6); and the $q(f) \rightarrow q(x)$ handoff is closed by a Gauss–Newton truncation of Price’s theorem, which is what makes the load-bearing $\partial/\partial S_i$ slot PSD by construction (§7).

Contents

1	Setup and ELBO road-map	1
2	The SING contract: exactly what EFGP must supply	2
3	Notation cheat-sheet	4
4	The $q(f)$ update: GP regression on latent velocities	4
4.1	Transition stripped to its f -content	4
4.2	Layer 1: GP regression with known x	4
4.3	Layer 2: uncertain x , in a finite basis	5
5	The EFGP basis choice	6
6	Computing the $q(x)$-expectations that build A_r, h_r	6
6.1	Backend 1: gmix closed-form spreader (GPU default)	7
6.2	Backend 2: analytic Taylor-envelope NUFFT (CPU default)	8
6.3	Backend 3: Monte Carlo cloud (reference)	8
7	The $q(x)$ natural-gradient step	9
8	Algorithm summary and complexity	14
A	Math-to-code dictionary	16

B V-Hutch restoration (opt-in)	16
B.1 Closed-form $\mathbb{E}_{q(x_i)}[V]$ via BTTB-diagonal sums of $DA^{-1}D$	16
B.2 Hutchinson estimator of ω_r	17
B.3 Per-transition gradient: exact in m , S -slot dropped	17
B.4 Homogeneous- S vs heterogeneous- S sub-modes	17
C Implementation and benchmarks	18
C.1 Custom-VJP shims: why a shim, not pure autodiff	18
C.2 Point-eval alternative: linearised-shim	18
C.3 Wall-time comparisons ($K = 10$, $T = 500$)	18
C.4 ∇V magnitude diagnostic	18
C.5 Op count for V-Hutch per outer iter	18

1 Setup and ELBO road-map

Latent SDE and observations

We assume an Euler-discretised latent SDE

$$x_{i+1} = x_i + \Delta_i f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \Delta_i \Sigma), \quad (1)$$

with Gaussian observations $p(y_i | x_i)$. Throughout, we abbreviate

$$\delta_i := x_{i+1} - x_i. \quad (2)$$

The diffusion is taken diagonal, $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_{D_x}^2)$. The drift coordinates carry independent GP priors,

$$f_r \sim \text{GP}(0, k_\theta), \quad r = 1, \dots, D_x. \quad (3)$$

Joint, ELBO, and the three vEM updates

The joint factorises as

$$p(f, x_{0:T}, y_{0:T}) = p(f) p(x_0) \prod_i p(x_{i+1} | x_i, f) \prod_i p(y_i | x_i), \quad (4)$$

with structured variational posterior $q(x_{0:T}) q(f)$. The ELBO is

$$\begin{aligned} \mathcal{L}(q) = & \underbrace{\mathbb{E}_{q(x)}[\log p(y | x)]}_{\substack{\text{Gaussian closed-form;} \\ q(x)\text{-grad: backbone autograd}}} + \underbrace{\sum_i \mathbb{E}_{q(x,f)}[\log p(x_{i+1} | x_i, f)]}_{\substack{\text{transition — couples } q(x), q(f); \\ q(x)\text{-grad: } \textit{hand-derived custom_vjp} \text{ (§7);} \\ q(f)\text{-update: closed-form } A_r \mu_r = h_r \text{ (§4)}}} \\ & + \underbrace{\mathbb{E}_{q(f)}[\log p(f) - \log q(f)]}_{\substack{\text{constant in } q(x); \\ \text{enters } q(f)\text{-update + M-step}}} + \underbrace{H[q(x)] + \mathbb{E}_{q(x_0)}[\log p(x_0)]}_{\substack{\text{Gaussian closed-form;} \\ q(x)\text{-grad: backbone autograd}}}. \end{aligned} \quad (5)$$

vEM cycles three coupled updates against this ELBO:

q(f) update (§4–§6): maximise $\mathcal{L}$ over $q(f)$ with $q(x)$ fixed. Closed-form Gaussian $q(w_r) = \mathcal{N}(\mu_r, A_r^{-1})$; $A_r \mu_r = h_r$ via Jacobi-PCG.

q(x) update (§7): SING ρ -damped natural gradient on $\mathcal{L}$ with $q(f)$ fixed. The transition contribution’s $q(x)$ -integrals are closed in x by Stein (for the pairwise piece) and a first-order Taylor linearisation at m_i (for the residual nonlinearity in $\bar{f}$ and V).

M-step (separate doc): kernel hypers θ maximise the collapsed marginal NLL. At the $q(f)$ -optimum,

$$\mathcal{L}_M(\theta) = -\frac{1}{2} \sum_r \text{Re} \langle h_{\theta,r}, \mu_{\theta,r} \rangle + \frac{D_{\text{out}}}{2} \log |A_\theta|. \quad (6)$$

Only the transition term in (5) couples $q(x)$ and $q(f)$ non-trivially; the rest of this document analyses that term.

Section map

§	Addresses	Produces
2	the SING contract	3 required moments, 6 VJP slots, departures D1–D3
3	—	Notation glossary
4 (incl. 4.1–4.3)	transition at fixed $q(x)$	A_r, h_r (basis-agnostic)
5	same	Fourier specialisation: BTTB + cheap Jacobian
6	same	the $q(x)$ -expectations that fill A_r, h_r
7 (incl. 7.7)	transition at fixed $q(f)$	per-transition $(\partial/\partial m_i, \partial/\partial S_i, \partial/\partial S_{i,i+1})$ via Stein + Taylor at m_i + gmix-gather drift moments
8	—	end-to-end pseudocode + complexity

2 The SING contract: exactly what EFGP must supply

SING’s backbone (algorithm and derivation: the SING paper) asks a drift model for *three* per-transition moments and nothing else. They are the methods of `sing.sde.SDE`, called at the current $q(x_i) = \mathcal{N}(m_i, S_i)$:

SDE method	math	shape	name in <code>trm</code>
<code>f(..., t, m, S)</code>	$\mathbb{E}_{q(x_i)}[f(x_i)]$	(D)	<code>Ef</code>
<code>ff(..., t, m, S)</code>	$\mathbb{E}_{q(x_i)}[\ f(x_i)\ ^2]$	scalar	<code>Eff</code>
<code>dfdx(..., t, m, S)</code>	$\mathbb{E}_{q(x_i)}[J_f(x_i)]$	(D, D)	<code>Edfdx</code>

The three land in `compute_neg_CE_single` (`sing/utils/sing_helpers.py:289--304`), which assembles the per-transition negative cross-entropy into a scalar `trm` (eq. 32 and §7). `nat_grad_transition` (`sing/sing.py:46`) then `vmap(jax.grad(...))`s that scalar w.r.t. the *mean* parameters $\mu = (\mathbb{E}x_i, \mathbb{E}x_i x_i^\top, \mathbb{E}x_i x_{i+1}^\top)$; by exponential-family duality that *is* the natural gradient on (J, h, L) , which SING ρ -damps and applies.

Two conventions to keep straight. (i) The diffusion enters `trm` as one scalar prefactor $-1/(2\Delta_i\sigma^2)$ (line 304): the code is the **isotropic** case $\Sigma = \sigma^2 I$ (`sigma_drift_sq` is a scalar), so the $\Sigma^{-1} / \sigma_r^{-2}$ written throughout this note is a single σ^{-2} in practice, and `ff` carries no Σ^{-1} inside. (ii) Lines 301–302 add the linear-input contribution Bu_i ; the second of these also multiplies `Ef`, so `Ef`’s cotangent picks up a $+2\Delta_i^2(Bu_i)$ term. Neither changes any statement below.

The contract is three values plus six VJP slots. Since SING only ever differentiates *through* the three returns, a drift backend is fully specified per transition by those three values and their VJPs w.r.t. (m_i, S_i) . With $\bar{f}_r(x) = \phi(x)^* \mu_r$ the $q(f)$ posterior-mean drift and g the upstream cotangent, EFGP supplies:

slot	value (how)	$\partial/\partial m_i$	$\partial/\partial S_i$	status
Ef	$\mathbb{E}_{q(x_i)}[\bar{f}]$ (gmix-gather)	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top g$	0	m exact; S GN-dropped
Eff	$\ \mathbb{E}_{q(x_i)}[\bar{f}]\ ^2$ (from Ef)	$2g \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \mathbb{E}_{q(x_i)}[\bar{f}]$	$g \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \mathbb{E}_{q(x_i)}[\bar{f}] \succeq 0$	GN + cov-drop
Edfdx	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ (gmix-gather)	0 (raw array)	0 (raw array)	GN-dropped

Code: `FrozenEFGPDrift` with `ef_with_jac_grad` and `eff_with_grads` (`sing/efgp_jax_drift.py:39--134`). Note **Eff**'s forward stores only $\|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$; the $\text{tr}[\cdot S_i]$ piece of the true value lives only in the $\partial/\partial S_i$ slot, which is all SING consumes.

Departures from the generic SING algorithm.

- D1 $q(f)$ is a Fourier-feature Gaussian updated in closed form.** Generic SING takes a fixed parametric drift; EFGP's drift is a GP whose weight posterior $q(w_r) = \mathcal{N}(\mu_r, A_r^{-1})$ solves $A_r \mu_r = h_r$ — an extra coordinate-ascent block interleaved with SING's $q(x)$ update (§4–6), plus a collapsed kernel M-step (`efgp_mstep.tex`).
- D2 The S -slots are Gauss–Newton truncated (§7).** Generic SING would let `jax.grad` carry the exact Price/Bonnet derivatives through an on-graph quadrature (e.g. `GaussHermiteQuadrature`). EFGP computes the moments off-graph (autodiff through NUFFTs inside the per-transition `vmap` is $\sim 50\times$; App. C) and injects the GN-truncated table above — which is what buys a PSD-by-construction $\partial/\partial S_i$ on the load-bearing slot.
- D3 `ff` carries only $\bar{f}$, not the $q(f)$ variance.** The exact $q(f)$ -average is $\mathbb{E}_{q(f)}[\|f(x)\|^2] = \|\bar{f}(x)\|^2 + V(x)$; production drops V and its gradient (`restore_qf_variance='none'`, “Approximation A”, §7 item (iv)). Opt-in restoration: App. B.

Everything else in the ELBO — emissions, $H[q(x)]$, initial — is Gaussian closed-form and handled unchanged by SING's backbone. A reader who wants only the interface can stop here.

3 Notation cheat-sheet

m_i, S_i	marginal mean and covariance of $q(x_i)$.
$S_{i,i+1}$	cross-covariance $\text{Cov}(x_i, x_{i+1})$.
$d_i := m_{i+1} - m_i$	mean increment.
$C_i := S_{i,i+1} - S_i =$	Stein cross-cov correction.
$\text{Cov}(x_i, x_{i+1} - x_i)$	
$\phi(x), \phi_\theta(x)$	finite basis for f ; subscript θ added when the kernel parameterisation matters (§5 onward).
ξ_k, D_θ	equispaced Fourier spectral grid and diagonal kernel-weight matrix (eq. 20).
A_r, h_r, μ_r	$q(f_r)$ precision, RHS, and posterior mean in the basis (eq. 17).
$\bar{f}_r(x) = \phi(x)^* \mu_r$	posterior mean drift under $q(f)$ (exact).
$\tilde{f}(x_i)$	first-order Taylor of $\bar{f}$ at m_i , the §7 linearisation.
$J_{\bar{f}}(x)$	Jacobian of $\bar{f}$ at x ; for Fourier features cheap (eq. 21).
$V_r(x) = \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x)$	$q(f)$ posterior variance contribution at x (eq. 41).
$\omega_r(\delta)$	BTTB-diagonal projection of $D_\theta A_r^{-1} D_\theta$ on lag δ (App. B, eq. 46).

4 The $q(f)$ update: GP regression on latent velocities

ELBO term: transition term $\sum_i \mathbb{E}_{q(x,f)} [\log p(x_{i+1} | x_i, f)] - \text{KL}[q(f) \| p(f)]$ at fixed $q(x)$. **Quantity produced:** closed-form Gaussian $q(w_r) = \mathcal{N}(\mu_r, A_r^{-1})$ via $A_r \mu_r = h_r$ (eq. 17).

4.1 Transition stripped to its f -content

The Euler transition log-density is

$$\log p(x_{i+1} | x_i, f) = -\frac{1}{2\Delta_i} [\delta_i - \Delta_i f(x_i)]^\top \Sigma^{-1} [\delta_i - \Delta_i f(x_i)] + \text{const.} \quad (7)$$

The constant and the $\delta_i^\top \Sigma^{-1} \delta_i$ term do not involve f . Expanding for diagonal Σ , the part of $\log p(x_{i+1} | x_i, f)$ that depends on coordinate f_r is

$$\boxed{\frac{1}{\sigma_r^2} \delta_{i,r} f_r(x_i) - \frac{\Delta_i}{2\sigma_r^2} f_r(x_i)^2.} \quad (8)$$

This is a Gaussian likelihood, quadratic in the function values $f_r(x_i)$. That is the *only* fact about the transition we will need. The $q(f_r)$ update reduces to combining (8) with the GP prior (3). Because Σ is diagonal and the f_r -priors are independent, the $q(f_r)$ updates decouple across r ; from now on we work coordinate-wise and suppress the index r when convenient.

4.2 Layer 1: GP regression with known x

Pretend, for this section only, that the latent path $X = \{x_0, \dots, x_{T-1}\}$ is observed. Define the pseudo-velocity

$$y_{i,r}^{\text{pseudo}} := \frac{\delta_{i,r}}{\Delta_i} = \frac{x_{i+1,r} - x_{i,r}}{\Delta_i}. \quad (9)$$

Completing the square in (8) shows the contribution from transition i is

$$-\frac{\Delta_i}{2\sigma_r^2} (y_{i,r}^{\text{pseudo}} - f_r(x_i))^2 + \text{const}, \quad (10)$$

i.e. a Gaussian observation of $f_r(x_i)$ with precision $\lambda_{i,r} = \Delta_i/\sigma_r^2$. Stack

$$f_{r,X} = [f_r(x_0), \dots, f_r(x_{T-1})]^\top, \quad [K_X]_{ij} = k_\theta(x_i, x_j), \quad \Lambda_r = \text{diag}(\lambda_{i,r}), \quad (11)$$

and let y_r^{pseudo} collect the pseudo-velocities. Then $q(f_{r,X}) = \mathcal{N}(m_{r,X}, C_{r,X})$ with the standard GP-regression identities

$$\boxed{C_{r,X}^{-1} = K_X^{-1} + \Lambda_r, \quad C_{r,X}^{-1} m_{r,X} = \Lambda_r y_r^{\text{pseudo}}.} \quad (12)$$

Template. *The $q(f)$ update is GP regression on latent velocities.* The remainder of this section modifies (12) to the case where x is uncertain; §5 then specialises the basis to make it FFT-cheap.

4.3 Layer 2: uncertain x , in a finite basis

In SING X is not observed; only $q(x_{0:T})$ is available, a Gaussian Markov variational posterior with marginals (m_i, S_i) and cross-covariances $S_{i,i+1}$. The K-space packaging (12) no longer applies directly — with X random, both $f_{r,X}$ and K_X are random — so rather than carry a function posterior we parametrise the drift directly by weights,

$$f_r(x) = \phi(x)^* w_r, \quad w_r \sim \mathcal{N}(0, I_M), \quad (13)$$

the basis chosen so the iid prior on w_r reproduces the GP prior on f_r . The coordinate-ascent VI update for $q(w_r)$ is then *just the expected log-joint as a function of w_r* :

$$\log q^*(w_r) = \mathbb{E}_{q(x)}[\log p(x_{0:T}, w_r)] + \text{const}, \quad (14)$$

the expectation over x only, with w_r the variable being updated. The only w_r -dependent pieces are the prior $\log p(w_r) = -\frac{1}{2} w_r^* w_r$ and the transition content (8) with $f_r(x_i) = \phi(x_i)^* w_r$ — so $\delta_{i,r} f_r(x_i) = \delta_{i,r} \phi(x_i)^* w_r$ is linear in w_r and $f_r(x_i)^2 = w_r^* \phi(x_i) \phi(x_i)^* w_r$ is quadratic. Hence

$$\log q^*(w_r) = -\frac{1}{2} w_r^* A_r w_r + \Re(h_r^* w_r) + \text{const}, \quad (15)$$

with

$$\boxed{A_r = I + \frac{1}{\sigma_r^2} \sum_i \Delta_i \mathbb{E}_{q(x_i)}[\phi(x_i) \phi(x_i)^*], \quad h_r = \frac{1}{\sigma_r^2} \sum_i \mathbb{E}_{q(x_i, x_{i+1})}[\phi(x_i) (x_{i+1,r} - x_{i,r})].} \quad (16)$$

The expectation over $q(x)$ touches only the features, contracting $\phi\phi^*$ and ϕ to the deterministic $M \times M$ matrix and M -vector in (16); w_r is never integrated. So $\log q^*(w_r)$ is exactly quadratic in w_r , and completing the square gives the Gaussian

$$q(w_r) = \mathcal{N}(\mu_r, A_r^{-1}), \quad A_r \mu_r = h_r. \quad (17)$$

Stein collapse of the linear term in h_r . The pairwise expectation in (16) under $q(x_i, x_{i+1})$ is awkward. Stein's lemma collapses it to marginal expectations plus a cross-cov correction. With $d_i := m_{i+1} - m_i$ and $C_i := S_{i,i+1} - S_i = \text{Cov}(x_i, x_{i+1} - x_i)$, for any smooth g ,

$$\mathbb{E}[g(x_i) (x_{i+1,r} - x_{i,r})] = \mathbb{E}[g(x_i)] d_{i,r} + \mathbb{E}[J_g(x_i)] C_{i,r}. \quad (18)$$

Apply with $g = \phi$ (componentwise) inside (16):

$$h_r = \frac{1}{\sigma_r^2} \sum_i \left\{ \underbrace{\mathbb{E}_{q(x_i)}[\phi(x_i)] d_{i,r}}_{\text{mean-increment}} + \underbrace{\mathbb{E}_{q(x_i)}[J_\phi(x_i)]^* C_{i,r}}_{\text{Stein correction}} \right\}. \quad (19)$$

Every quantity in (17) now depends only on *marginal* expectations under $q(x_i)$, plus the cross-cov correction C_i .

5 The EFGP basis choice

ELBO term: still the transition term at fixed $q(x)$. **Quantity produced:** Fourier basis specialisation of (17): A_r becomes BTTB-Toeplitz and J_ϕ is frequency multiplication.

EFGP specialises the basis ϕ in (13) to Fourier features:

$$\phi_\theta(x) = D_\theta F_x, \quad F_x[k] = e^{2\pi i x^\top \xi_k}, \quad \phi_\theta(x)^* \phi_\theta(x') \approx k_\theta(x, x'), \quad (20)$$

with $\{\xi_k\}_{k=1}^M$ an equispaced spectral grid and D_θ diagonal with square-root spectral weights chosen to match k_θ . This is the only kernel approximation introduced. The system (17) is unchanged in form; what we gain are two structural simplifications.

Cheap Jacobian. Differentiating (20),

$$\frac{\partial \phi_k(x)}{\partial x_j} = 2\pi i \xi_{k,j} \phi_k(x), \quad (21)$$

so J_ϕ is just frequency-multiplication of ϕ . The Stein correction in (19) costs no more than evaluating ϕ itself.

BTTB structure of $\mathbb{E}_{q(x_i)}[\phi\phi^*]$. For Fourier features, $\phi(x)\phi(x)^*$ has (k, ℓ) entry $D_{\theta,k} e^{2\pi i x^\top (\xi_\ell - \xi_k)} D_{\theta,\ell}$. The expectation under $q(x_i)$ depends on $\xi_\ell - \xi_k$ only:

$$(\mathbb{E}_{q(x_i)}[\phi\phi^*])_{k\ell} = D_{\theta,k} \mathbb{E}_{q(x_i)}[e^{2\pi i x_i^\top (\xi_\ell - \xi_k)}] D_{\theta,\ell}. \quad (22)$$

On an equispaced grid this makes $\mathbb{E}_{q(x_i)}[\phi\phi^*]$ Block-Toeplitz with Toeplitz Blocks (BTTB), so $A_r v$ costs $O(M \log M)$ via FFT instead of $O(M^2)$.

6 Computing the $q(x)$ -expectations that build A_r, h_r

ELBO term: still the transition term at fixed $q(x)$. **Quantity produced:** the marginal expectations $\mathbb{E}_{q(x_i)}[\phi(x_i)\phi(x_i)^*]$, $\mathbb{E}_{q(x_i)}[\phi(x_i)]$, $\mathbb{E}_{q(x_i)}[J_\phi(x_i)]$ that fill (17)–(19).

Because every $q(x_i) = \mathcal{N}(m_i, S_i)$ is Gaussian and every ϕ_k is a complex exponential, each marginal expectation appearing in (16)–(19) is the Gaussian characteristic function evaluated at frequency differences:

$$\mathbb{E}_{q(x_i)}[e^{2\pi i x_i^\top \xi}] = e^{2\pi i m_i^\top \xi} e^{-2\pi^2 \xi^\top S_i \xi}, \quad (23)$$

a phase at m_i times a Gaussian envelope set by S_i . So *every* quantity needed in (17)–(19) is one **Gaussian-enveloped non-uniform Fourier sum**

$$F(\xi) = \sum_i w_i e^{2\pi i m_i^\top \xi} e^{-2\pi^2 \xi^\top S_i \xi}, \quad (24)$$

evaluated on the frequency-*difference* grid (for the BTTB generator of A_r) or on the spectral grid (for h_r), with w_i the relevant Stein weight (Δ_i/σ^2 , $d_{i,r}/\sigma^2$, or $[C_i]_{j,r}/\sigma^2$; assembled by `flatten_stein`). Three interchangeable backends compute (24), differing only in how the *per-source* envelope S_i is handled. `estep_method='auto'` (the default) resolves by device:

<code>estep_method</code>	envelope handling	cost	error
<code>'gmix'</code> (<i>default on GPU</i>)	each $\mathcal{N}(m_i, S_i)$ tabulated on a fine spatial grid over a $(2r+1)^D$ stencil, then FFT	$O(N r^D + N_f^D \log N_f)$	stencil tail truncation (n_σ)
<code>'analytic'</code> (<i>default on CPU; D=2 only</i>)	$S_i = \bar{S} + \Delta S_i$; Taylor $e^{-2\pi^2 \xi^\top \Delta S_i \xi}$ to order p	$O(P(p) (N + M \log M))$	ΔS truncation; <i>exact</i> at $\Delta S=0$
<code>'mc'</code> (<i>reference</i>)	S marginal samples per source, plain NUFFT	$O(NS + M \log M)$	Monte Carlo variance

Resolution logic (`efgp_em.py:698--712`): `'gpu' → 'gmix'`, otherwise `'analytic'`, falling back to `'gmix'` when $D \neq 2$. The devices disagree because the bottleneck does: on CPU the `gmix` per-source spread dominates the wall, so `'analytic'` is $\sim 6\times$ faster on the $q(f)$ update; on GPU scatter and `cuFFT` are nearly free (the wall is JIT compile plus the sequential-in- T block-tridiagonal smoother `lax.scan`), so `'analytic'` only adds `cuFINUFFT` launch overhead. Any explicit value overrides `'auto'`.

6.1 Backend 1: gmix closed-form spreader (GPU default)

Substituting into A_r (16) preserves BTTB structure: the generating kernel on the difference grid $\Delta \xi_k$ is the Type-1 NUFFT of weighted Gaussians *centred at m_i with per-source widths S_i* . Operationally this replaces the fixed NUFFT spreading kernel by a per-source $\mathcal{N}(\cdot; m_i, S_i)$ on a truncated stencil; the FFT and deconvolve steps are unchanged. The RHS terms in (19) receive the same treatment.

The companion note `efgp_estep_charfun.tex` carries the full derivation, the explicit closed-form generator, complexity proofs, and the bucketed / Gauss–Hermite alternatives considered. The closed-form spreader eliminates Monte Carlo variance at the cost of a per-source spread; in single-scale regimes the asymptotic cost matches a standard NUFFT, and heterogeneous S_i costs an extra $\propto (\sigma_{\max}/\sigma_{\min})^D$ factor. The stencil is truncated at n_σ standard deviations (default 1.5), which leaves a tail bias present *even at $\Delta S = 0$* ($\sim 10\%$ vs. MC on the reference problem) — the accuracy motivation for Backend 2. Code: `compute_mu_r_gmix_jax` (`efgp_jax_drift.py:269`), spreader in `sing/efgp_gmix_spreader.py`.

6.2 Backend 2: analytic Taylor-envelope NUFFT (CPU default)

Split the per-source covariance about the trajectory mean, $S_i = \bar{S} + \Delta S_i$, and Taylor-expand *only* the deviation factor:

$$e^{-2\pi^2 \xi^\top S_i \xi} = e^{-2\pi^2 \xi^\top \bar{S} \xi} e^{u_i(\xi)}, \quad u_i(\xi) := -2\pi^2 \xi^\top \Delta S_i \xi, \quad e^{u_i} \approx \sum_{k \leq p} \frac{u_i^k}{k!}. \quad (25)$$

Each term of u_i^k factorises into a ξ -monomial times a ΔS_i -monomial, so (24) becomes a fixed number of *ordinary* type-1 NUFFT on the same nodes $\{m_i\}$ with re-weighted source vectors:

$$F(\xi) \approx e^{-2\pi^2 \xi^\top \bar{S} \xi} \sum_{\text{pairs}} c_{\text{pair}}(\xi) \text{NUFFT1}(\{m_i\}, \{w_i \nu_{\text{pair},i}\})(\xi), \quad (26)$$

with the $(c_{\text{pair}}, \nu_{\text{pair}})$ enumerated by `_env_taylor_pairs (efgp-jax-drift.py:525)`: $P(p) = \binom{p+2}{2}$ pairs in 2-D, i.e. 1, 4, 10 at `analytic_order` $p = 0, 1, 2$. All pairs share the nodes $\{m_i\}$ and are evaluated as ONE batched transform (a single FINUFFT plan), so order $\rightarrow$ order cost is nearly flat; the same expansion is applied on the difference grid (BTTB generator) and on the spectral grid (RHS). Code: `compute_mu_r_analytic_jax (efgp-jax-drift.py:569)`.

No fine grid and no stencil: *exact* for homogeneous S (any order), with the truncation error controlled by the S -spread rather than by a grid. Order 1 holds to $<4\%$ up to $\sim 20\times$ spread in the S_i eigenvalues, order 2 to $\sim 60\times$; converged fits sit near $1.4\times$. 2-D only in v0. Full derivation and benchmarks: `efgp_analytic_qf.tex`.

6.3 Backend 3: Monte Carlo cloud (reference)

The same expectations can be evaluated by Monte Carlo. For each $i = 0, \dots, T-2$, draw S marginal samples $x_i^{(s)} \sim q(x_i)$ and stack as $X = \{x_i^{(s)}\}_{i,s} \in \mathbb{R}^{(T-1)S \times D_x}$. With

$$[F_X]_{k,(i,s)} = e^{2\pi i (x_i^{(s)})^\top \xi_k}, \quad \phi_\theta(x_i^{(s)}) = D_\theta(F_X)_{\cdot,(i,s)}, \quad (27)$$

and per-sample weights $(W_r)_{(i,s)} = \Delta_i / (S\sigma_r^2)$, A_r becomes

$$A_r \approx I + D_\theta T_r D_\theta, \quad T_r := F_X^* W_r F_X, \quad (28)$$

with $(T_r)_{k\ell}$ depending on $\xi_\ell - \xi_k$ only. T_r is BTTB on the equispaced grid; a single Type-1 NUFFT over X produces its generating kernel, and $A_r v = v + D_\theta(T_r(D_\theta v))$. The system $A_r \mu_r = h_r$ is solved by Jacobi-preconditioned CG.

For h_r , set

$$(a_r)_{(i,s)} = \frac{d_{i,r}}{S\sigma_r^2}, \quad (c_{j,r})_{(i,s)} = \frac{[C_i]_{j,r}}{S\sigma_r^2}, \quad (29)$$

so (19) estimates as

$$h_{1,r} \approx D_\theta F_X^* a_r, \quad h_{2,r} \approx \sum_{j=1}^{D_x} (-2\pi i \xi_j) \odot D_\theta F_X^* c_{j,r}, \quad h_r = h_{1,r} + h_{2,r}. \quad (30)$$

The factor $-2\pi i \xi_j$ in $h_{2,r}$ (rather than $+2\pi i \xi_j$) comes from the adjoint Jacobian J_ϕ^* : $\overline{2\pi i \xi_{k,j} \phi_k(x)} = -2\pi i \xi_{k,j} \overline{\phi_k(x)}$.

The Stein form (19) can be checked against a naive pair-sampling estimator (draw transition pairs $(x_i^{(s)}, x_{i+1}^{(s)}) \sim q(x_i, x_{i+1})$ and use sampled pseudo-velocities in (12)); both estimators target the same h_r , with Stein strictly lower variance. SING uses Stein in production and pair-sampling only as a regression test.

7 The $q(x)$ natural-gradient step

ELBO term: transition term at fixed $q(f) = \sum_i \mathbb{E}_{q(x_i, x_{i+1})} [\mathbb{E}_{q(f)} [\log p(x_{i+1} | x_i, f)]]$. **Quantity produced:** per-transition $(\partial/\partial m_i, \partial/\partial S_i, \partial/\partial S_{i,i+1})$ of this term, ready for SING's ρ -damped natural-grad update on $q(x)$.

Setup

SING does ρ -damped natural gradient on $q(x)$ (constrained Gaussian-Markov), using

$$\tilde{\nabla}_\eta \mathcal{L} = \partial \mathcal{L} / \partial \mu, \quad \eta = (J, h, L), \quad \mu = (\mathbb{E}[x_i], \mathbb{E}[x_i x_i^\top], \mathbb{E}[x_i x_{i+1}^\top]), \quad (31)$$

i.e. exponential-family duality (§2). Mean params re-parameterise cleanly to $(m_i, S_i, S_{i,i+1})$. This section derives *why* the six VJP slots of §2 take the values they do — departure **D2**.

The transition term, and how the code assembles it

At fixed $q(f)$, with $\bar{f}_r(x) = \phi(x)^* \mu_r$ deterministic but nonlinear in x and $V_r(x) = \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x)$ the $q(f)$ posterior variance contribution, the per-transition piece is

$$\begin{aligned} \mathbb{E}_{q(x_i, x_{i+1})} [\mathbb{E}_{q(f)} [\log p(x_{i+1} | x_i, f)]] &= - \frac{\mathbb{E}[\delta_i^\top \Sigma^{-1} \delta_i]}{2\Delta_i} + \mathbb{E}_{q(x_i, x_{i+1})} [\delta_i^\top \Sigma^{-1} \bar{f}(x_i)] \\ &\quad - \frac{\Delta_i}{2} \mathbb{E}_{q(x_i)} [\bar{f}^\top \Sigma^{-1} \bar{f} + V] + \text{const.} \end{aligned} \quad (32)$$

This is assembled into the scalar `trm` of `compute_neg_CE_single` (`sing_helpers.py:289--304`) from four pieces:

piece	math	where <code>trm</code> is built (lines in <code>compute_neg_CE_single</code>)
(i)	$\mathbb{E}[\delta^\top \Sigma^{-1} \delta] / (2\Delta_i)$	294-296 (traces of m, S, SS)
(ii)	$\mathbb{E}_{q(x_i, x_{i+1})} [\delta^\top \Sigma^{-1} \bar{f}]$	298-299 (<code>Ef</code> and <code>Edfdx</code>)
(iii)	$\mathbb{E}_{q(x_i)} [\bar{f}^\top \Sigma^{-1} \bar{f}]$	297 (<code>(del_t)**2 * Eff</code>)
(iv)	$\mathbb{E}_{q(x_i)} [V]$	dropped (production)

The drift moments `Ef`, `Eff`, `Edfdx` consumed in lines 297-299 are pre-computed off-graph by `drift_moments_gmix_jax` and exposed via the `FrozenEFGPDrift` pytree — from `compute_neg_CE_single`’s view they are per-transition constants. The Taylor framework below dictates how their (m_t, S_t) -gradients are filled in via `custom.vjp`; the four item subsections then walk each piece’s value and gradient flow.

The Gauss-Newton framework

Items (ii)-(iv) need (m_i, S_i) -derivatives of expectations under $q(x_i) = \mathcal{N}(m_i, S_i)$ of nonlinear functions $(\bar{f}, J_{\bar{f}}, \bar{f}^\top \Sigma^{-1} \bar{f}, V)$. Production handles all of these under a single principled approximation: a Gauss-Newton (GN) truncation of Price’s theorem.

Price’s theorem (Bonnet’s lemma). For any sufficiently smooth g and Gaussian $q(x_i)$,

$$\partial_{m_i} \mathbb{E}_{q(x_i)} [g(x_i)] = \mathbb{E}_{q(x_i)} [\nabla g(x_i)], \quad \partial_{S_i} \mathbb{E}_{q(x_i)} [g(x_i)] = \frac{1}{2} \mathbb{E}_{q(x_i)} [\nabla^2 g(x_i)]. \quad (33)$$

These are exact identities, expressing the (m, S) -derivatives of a Gaussian expectation as expectations of the gradient and Hessian of the integrand.

The GN move on the load-bearing slot. For $g = \bar{f}^\top \Sigma^{-1} \bar{f}$ (the quadratic in item (iii)), the Hessian splits as

$$\nabla^2(\bar{f}^\top \Sigma^{-1} \bar{f}) = 2(\underbrace{J_{\bar{f}}^\top \Sigma^{-1} J_{\bar{f}}}_{\text{GN piece: PSD}} + \underbrace{\sum_r \sigma_r^{-2} \bar{f}_r H_{\bar{f}_r}}_{\text{residual: sign-indefinite}}). \quad (34)$$

The first piece is PSD-by-construction; the second (the residual Hessian) is sign-indefinite because $\bar{f}_r$ has no sign constraint. **The Gauss-Newton approximation drops the residual Hessian** — the standard move for sum-of-squares objectives. Plugged into Price:

$$\begin{aligned} \partial_{S_i} \mathbb{E}[\bar{f}^\top \Sigma^{-1} \bar{f}] &\stackrel{\text{Price}}{=} \mathbb{E}[J_{\bar{f}}^\top \Sigma^{-1} J_{\bar{f}}] + \frac{1}{2} \mathbb{E}[\sum_r \sigma_r^{-2} \bar{f}_r H_{\bar{f}_r}] \stackrel{\text{GN}}{\approx} \mathbb{E}[J_{\bar{f}}^\top \Sigma^{-1} J_{\bar{f}}] \\ &\stackrel{\text{Cov-drop}}{\approx} \mathbb{E}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}[J_{\bar{f}}] \succeq 0. \end{aligned} \quad (35)$$

The second approximation (the covariance drop) replaces $\mathbb{E}[J^\top J]$ with $\mathbb{E}[J]^\top \mathbb{E}[J]$ — also second-order in the nonlinearity of $\bar{f}$. The result is PSD-by-construction, a structural consequence of the GN form rather than a coincidence.

The same GN move handles every other truncation. For the linear pieces, Price + GN gives

$$\partial_{S_i} \mathbb{E}[\bar{f}] \stackrel{\text{Price}}{=} \frac{1}{2} \mathbb{E}[H_{\bar{f}}] \stackrel{\text{GN}}{\approx} 0, \quad \partial_{m_i} \mathbb{E}[J_{\bar{f}}] \stackrel{\text{Price}}{=} \mathbb{E}[H_{\bar{f}}] \stackrel{\text{GN}}{\approx} 0, \quad \partial_{S_i} \mathbb{E}[J_{\bar{f}}] \stackrel{\text{Price}}{=} \frac{1}{2} \mathbb{E}[\nabla^2 J_{\bar{f}}] \stackrel{\text{GN}}{\approx} 0. \quad (36)$$

Every dropped slot is a residual Hessian (of $\bar{f}$ or of $J_{\bar{f}}$) — the same object GN drops in the quadratic case. The cross-term gradient $\partial_{m_i} \mathbb{E}[\bar{f}^\top \Sigma^{-1} \bar{f}] = 2\mathbb{E}[J^\top \Sigma^{-1} \bar{f}]$ gets a further covariance drop in production: $2\mathbb{E}[J]^\top \Sigma^{-1} \mathbb{E}[\bar{f}]$.

Gradient table (GN-truncated Price’s identities).

quantity	value	$\partial/\partial m_i$	$\partial/\partial S_i$
$\mathbb{E}_{q(x_i)}[\bar{f}]$	$\mathbb{E}_{q(x_i)}[\bar{f}]$	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$	0 (GN: $\frac{1}{2}\mathbb{E}[H_{\bar{f}}]$ dropped)
$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$	0 (GN: $\mathbb{E}[H_{\bar{f}}]$ dropped)	0 (GN)
$\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$	$\mathbb{E}_{q(x_i)}[\bar{f}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}] + \text{tr}[\mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[J_{\bar{f}}] S_i]$	$2\mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]$	$\mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[J_{\bar{f}}] \succeq 0$

Why this is justified. GN is a well-studied Hessian approximation for sum-of-squares objectives: drop the residual Hessian, keep the PSD outer-product piece. Exact when residuals vanish ($\bar{f}_r \rightarrow 0$); accurate when $\|\bar{f}_r\| \|H_{\bar{f}_r}\| \ll \|J_{\bar{f}}\|^2$. The covariance drops $\text{Cov}(J, J)$ and $\text{Cov}(J, \bar{f})$ are of the same order. This is the same approximation underlying variational Gauss-Newton (Khan & Lin, 2017) and drift-linearisation in variational SDE inference (Oppen & Archambeau, 2009).

Why we can’t just autodiff. The exact $\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$ closes via spectral autocorrelation of $\mu_r D_\theta$,

$$\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}] = \sum_r \sigma_r^{-2} \sum_{k,\ell} \mu_{r,k}^* D_k D_\ell \mu_{r,\ell} e^{2\pi i m_i^\top \delta_{k\ell}} e^{-2\pi^2 \delta_{k\ell}^\top S_i \delta_{k\ell}}, \quad \delta_{k\ell} := \xi_\ell - \xi_k, \quad (37)$$

but ∂_{S_i} of this is a sign-indefinite sum of $\delta\delta^\top$ matrices (the full Hessian contribution to Price) — it would break the PSD property SING’s J -update relies on. The GN-truncated form (35) is what gives the load-bearing $\partial_{S_i} \succeq 0$. Separately, `jax.grad` through the spectral closed forms costs $\sim 50\times$ per inner iter under `vmop(grad(...))` (App. C).

Production base values via gmix-gather. The expectations $\mathbb{E}_{q(x_i)}[\bar{f}]$ and $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ in the table slots are computed exactly in closed form via the Gaussian char-fun (23): with $\bar{f}_r(x) = \phi(x)^* \mu_r$,

$$\mathbb{E}_{q(x_i)}[\bar{f}_r] = \sum_k \mu_{r,k} D_{\theta,k} e^{2\pi i m_i^\top \xi_k} e^{-2\pi^2 \xi_k^\top S_i \xi_k},$$

analogously $\mathbb{E}_{q(x_i)}[\partial_j \bar{f}_r]$ with frequency modulation by $(2\pi i \xi_{k,j})$. The gmix-gather primitive in `drift_moments_gmix_jax` (`efgp_jax.drift.py:793--870`; Type-2 analog of the §6.1 spreader: one batched IFFT + per-source Gaussian stencil gather) delivers (`Ef`, `Eff`, `Edfdx`) at all T base points in one shot.

Implementation: GN-as-custom_vjp. The GN choice maps operationally to *treat* $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ as constant w.r.t. (m_i, S_i) in *jax.grad*’s chain rule. Implemented in `FrozenEFGPDrift` via three mechanisms:

- `Edfdx` ($= \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$) exposed as a raw array, *no* `custom_vjp`; chain rule through `Edfdx` returns 0. This implements the GN drops $\partial_m \mathbb{E}[J_{\bar{f}}] = \mathbb{E}[H_{\bar{f}}] \rightarrow 0$ and $\partial_S \mathbb{E}[J_{\bar{f}}] \rightarrow 0$.
- `Ef` ($= \mathbb{E}_{q(x_i)}[\bar{f}]$) through `ef_with_jac_grad`’s `custom_vjp`: backward returns $\partial \text{Ef} / \partial m = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top g$, $\partial \text{Ef} / \partial S = 0$. The ∂_m injection coincides with the exact Price value $\partial_m \mathbb{E}_{q(x_i)}[\bar{f}] = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ (Price reduces to a single gradient term here, no Hessian to drop); the $\partial_S = 0$ drops $\frac{1}{2} \mathbb{E}[H_{\bar{f}}]$, the GN move.
- `Eff` ($= \|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$) through `eff_with_grads`’s `custom_vjp`: backward returns the GN entries directly, $\partial_m = 2g(\mathbb{E}_{q(x_i)}[\bar{f}] \cdot \mathbb{E}_{q(x_i)}[J_{\bar{f}}])$ and $\partial_S = g \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top \mathbb{E}_{q(x_i)}[J_{\bar{f}}]$. The forward stores only $\|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$; the tr-piece is absorbed into the gradient (saves a per-source $D \times D$ contraction, since SING uses the gradient not the value).

Strict-Taylor as a special case. Replacing $\bar{f}$ by its first-order Taylor expansion at m_i ,

$$\tilde{f}(x_i) := \bar{f}(m_i) + J_{\bar{f}}(m_i)(x_i - m_i), \quad (38)$$

with $J_{\bar{f}}(m_i)$ treated as constant w.r.t. (m_i, S_i) , gives the same gradient table by a different route. The two derivations are equivalent; the GN derivation is cleaner because (a) it doesn’t introduce a surrogate, (b) the PSD property of the load-bearing slot is structural, and (c) it connects to standard variational-GN literature.

Item (i): the δ -quadratic — exact

Math. $\mathbb{E}[\delta^\top \Sigma^{-1} \delta] / (2\Delta_i)$, a Gaussian quadratic in $\delta = x_{i+1} - x_i$ under $q(x_i, x_{i+1})$ — closed-form in $(m_i, m_{i+1}, S_i, S_{i+1}, S_{i,i+1})$.

Value (code, lines 294-296).

```
trm = trace(St_next + outer(mt_next, mt_next))
trm += trace(St + outer(mt, mt))
trm += -2 * trace(SS + outer(mt_next, mt))
```

Gradients. Pure `jax.grad` autodiff. No `custom_vjp`. SING-backbone, not EFGP-specific.

Item (ii): the Stein cross-term — value exact; gradients mix exact and GN-truncated pieces

Math. The Stein identity from §4.3, applied with $g = \bar{f}$, collapses the pairwise expectation *exactly* into a mean-increment piece and a Jacobian (Stein-correction) piece:

$$\mathbb{E}_{q(x_i, x_{i+1})}[\delta_i^\top \Sigma^{-1} \bar{f}(x_i)] = \underbrace{d_i^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]}_{\text{Part A: mean-increment}} + \underbrace{\text{tr}[\Sigma^{-1} \mathbb{E}_{q(x_i)}[J_{\bar{f}}] C_i^\top]}_{\text{Part B: Jacobian}}, \quad (39)$$

with $d_i := m_{i+1} - m_i$ and $C_i := S_{i,i+1} - S_i$ as before. The marginals $\mathbb{E}_{q(x_i)}[\bar{f}]$, $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ are computed by gmix-gather in closed form (framework above) and stored as `Ef`, `Edfdx` on `FrozenEFGPDrift`.

Value (code, lines 298-299).

```
trm += -2 * del_t * trace(outer(Ef, mt_next) + Edfdx @ SS.T)
trm += 2 * del_t * trace(outer(Ef, mt) + Edfdx @ St)
```

The outer-product terms are Part A in code form (combining to $-2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}]^\top d_i$); the `Edfdx @ ...` terms are Part B ($-2\Delta_i \text{tr}(\mathbb{E}_{q(x_i)}[J_{\bar{f}}] C_i^\top)$). The values themselves are exact.

Gradients. `jax.grad` of `trm` assembles three contributions per slot: *direct* (autodiff through the outer products and traces of lines 298–299, holding `Ef/Edfdx` constant — exact); *Ef-chain* (`ef_with_jac_grad`’s injections $\partial_m \text{Ef} = \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top g$, exact by Price, and $\partial_S \text{Ef} = 0$, the GN drop of $\frac{1}{2} \mathbb{E}[H_{\bar{f}}]$ — exactly this would be the non-zero envelope-derivative $-2\pi^2 \sum_k \mu_{r,k} D_k \xi_k \xi_k^\top e^{2\pi i m^\top \xi_k} e^{-2\pi^2 \xi_k^\top S \xi_k}$); and *Edfdx-chain* (a raw constant array, so all its slots are 0 — the “ $\mathbb{E}_{q(x_i)}[J]$ held constant” rule). All values and gradients use the *Gaussian-averaged* $\mathbb{E}_{q(x_i)}[\bar{f}]$, $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ from gmix-gather, not point-evals at m_i . Resolving per slot, each contribution labelled exact / GN-truncated:

slot	direct	Ef-chain	Edfdx-chain
∂m_t	$2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}]$ (<i>exact</i>)	$-2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top d_i$ (<i>exact</i>)	0 (<i>GN: $\mathbb{E}[H_{\bar{f}}]$ dropped</i>)
∂m_{t+1}	$-2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}]$ (<i>exact</i>)	0 (no m_{t+1} in <code>Ef</code>)	0 (<code>Edfdx</code> indep. of m_{t+1} ; exact)
∂S_t	$2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top$ (<i>exact, from $\text{tr}(\text{Edfdx } S_t)$</i>)	0 (<i>GN: env-deriv dropped</i>)	0 (<i>GN: env-deriv dropped</i>)
$\partial S_{i,i+1}$	$-2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top$ (<i>exact</i>)	0 (no $S_{i,i+1}$)	0 (<code>Edfdx</code> indep. of $S_{i,i+1}$; exact)

Net contributions of (ii) to ∂trm :

$$\begin{aligned} \partial m_t &= 2\Delta_i (\mathbb{E}_{q(x_i)}[\bar{f}] - \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top d_i), & \partial m_{t+1} &= -2\Delta_i \mathbb{E}_{q(x_i)}[\bar{f}], \\ \partial S_t &= 2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top, & \partial S_{i,i+1} &= -2\Delta_i \mathbb{E}_{q(x_i)}[J_{\bar{f}}]^\top. \end{aligned}$$

Status per slot: ∂m_{t+1} and $\partial S_{i,i+1}$ are *fully exact*; ∂m_t has Part A’s contribution exact but Part B’s GN-truncated ($\mathbb{E}[H_{\bar{f}}]$ dropped); ∂S_t has the direct piece exact but both chain contributions GN-truncated. Linear in $\bar{f}$; no PSD constraint applies (the PSD slot is item (iii)’s ∂S_t).

EFGP vs. SING. This is departure **D2** (§2) in detail: `SING`’s `nat_grad_transition` accepts whatever the backend’s (`f`, `ff`, `dfdx`) return, so an on-graph Gauss–Hermite backend would give `jax.grad` the exact $\partial_S \mathbb{E}_{q(x_i)}[\bar{f}]$; `EFGP` goes off-graph to avoid autodiff through NUFFT’s (App. C) and pays for it with the GN drops above.

Item (iii): the marginal quadratic — the GN slot

Math. $\mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$. Apply Price’s theorem and the GN move (framework above): the ∂_S derivative is the residual-Hessian decomposition (34), with the sign-indefinite residual piece dropped and the covariance further dropped, yielding (35). This is the load-bearing PSD-by-construction slot of the natural-grad. The exact spectral-autocorrelation form (37) is the alternative the GN framework rejected (sign-indefinite ∂_S).

Value (code, line 297). `trm += (del_t)**2 * Eff`, where `Eff =  $\|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$` is stored as `(Ef * Ef).sum(axis=-1)` (`drift_moments_gmix_jax` line 869). The forward holds only the $\mathbb{E}_{q(x_i)}[\bar{f}]^\top \mathbb{E}_{q(x_i)}[\bar{f}]$ piece of (35); the tr-piece is absorbed into the gradient by the `custom_vjp` below (saves a per-source contraction). A closed form for the exact $\mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}]$ exists — `gmix_E.full.Eff` (`sing/efgp_gmix_qx_moments.py:88`) — but is *not* called by the production fit; only `precompute_aux` from that module is, and only to supply `GmixQxAux` to the V-Hutch path.

Gradients. `Eff` comes from `eff_with_grads(m, S, eff_const, ef_const, jac_const)` (`efgp_jax_drift.py:57--75`); the `custom_vjp` backward returns

$$\partial \text{Eff} / \partial m_t = 2g(\text{ef_const} \cdot \text{jac_const}), \quad \partial \text{Eff} / \partial S_t = g(\text{jac_const}^\top \text{jac_const}),$$

with g the upstream cotangent ($= (\Delta_i)^2$ from line 297), `ef_const` = $\mathbb{E}_{q(x_i)}[\bar{f}]$, `jac_const` = $\mathbb{E}_{q(x_i)}[J\bar{f}]$. Reading these against the GN-truncated Price’s identities:

- ∂m_t injection $2g \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[\bar{f}]$. By Price, $\partial_m \mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}] = 2\mathbb{E}[J^\top \Sigma^{-1} \bar{f}]$; production drops $\text{Cov}(J, \bar{f})$ giving $2\mathbb{E}_{q(x_i)}[J]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[\bar{f}]$. **Covariance drop** (one of the GN approximations).
- ∂S_t injection $g \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[J\bar{f}]$. By Price, $\partial_S \mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}]$ has the residual-Hessian decomposition (34); GN drops the residual + $\text{Cov}(J, J)$, leaving $\mathbb{E}_{q(x_i)}[J]^\top \Sigma^{-1} \mathbb{E}_{q(x_i)}[J] \succeq 0$. **GN move on the load-bearing slot.**

Without the `custom_vjp`, autodiff would return zero in both slots (the forward `Eff` has no JAX-traced (m, S) -dependence). Net contributions of (iii) to ∂trm :

$$\partial m_t = 2(\Delta_i)^2 \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[\bar{f}], \quad \partial S_t = (\Delta_i)^2 \mathbb{E}_{q(x_i)}[J\bar{f}]^\top \mathbb{E}_{q(x_i)}[J\bar{f}] \succeq 0.$$

This is the GN slot: m -gradient from Part A’ (exact slot), S -gradient from Part B’ (Taylor’s trace piece). The Part B’ ∂S_t is the load-bearing PSD slot that protects SING’s ρ -damped J -update.

Item (iv): the V term — production drops; App. B opt-in via Price + Hessian-drop

Math. $\mathbb{E}_{q(x_i)}[V(x_i)]$, where $V(x) = \sum_r \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x)$ is the $q(f)$ posterior variance contribution. From

$$\text{Cov}_{q(f)}[f_r(x), f_r(x')] = \phi(x)^* A_r^{-1} \phi(x'), \quad (40)$$

the full $q(f)$ second moment of f splits as

$$\mathbb{E}_{q(f)}[f(x)^\top \Sigma^{-1} f(x)] = \sum_r \sigma_r^{-2} [\bar{f}_r(x)^2 + V_r(x)]; \quad (41)$$

V is the piece that (iii)’s $\bar{f}^\top \Sigma^{-1} \bar{f}$ doesn’t capture. V has no natural Part A/B split — it’s a single integral.

Production: value and gradients = 0. The $-\frac{\Delta_i}{2}\mathbb{E}_{q(x_i)}[V(x_i)]$ contribution is omitted from `trm`. Geometric justification: translation-invariance of stationary kernels gives $\nabla V(x) \approx 0$ wherever the source set $\{m_i^{(k)}\}$ looks isotropic at scale ℓ around x ; for SDE-recovery with diverse ICs and ergodic mixing this holds at every interior m_i , so the dropped $-\frac{\Delta_i}{2}\mathbb{E}_{q(x_i)}[\nabla V]$ contribution to the $q(x)$ gradient is negligible. Fails at boundary/edge points and cold-start.

App. B opt-in: Price + Hessian-drop. Apply Price’s theorem (33) to $g = V$:

$$\partial_{m_i}\mathbb{E}_{q(x_i)}[V] = \mathbb{E}[\nabla V], \quad \partial_{S_i}\mathbb{E}_{q(x_i)}[V] = \frac{1}{2}\mathbb{E}[\nabla^2 V], \quad (42)$$

and apply the same Hessian-drop the framework uses elsewhere ($\mathbb{E}[\nabla^2 V] \rightarrow 0$). and drop *only* the ∂_S Hessian, keeping the value and m -gradient exact:

$$\mathbb{E}_{q(x_i)}[V] \text{ (exact)}, \quad \partial_{m_i}\mathbb{E}_{q(x_i)}[V] = \mathbb{E}_{q(x_i)}[\nabla V] \text{ (exact)}, \quad \partial_{S_i}\mathbb{E}_{q(x_i)}[V] = \frac{1}{2}\mathbb{E}_{q(x_i)}[\nabla^2 V] \rightarrow 0. \quad (43)$$

This is exactly the item-(ii) $\bar{f}$ treatment: $\mathbb{E}_{q(x_i)}[V]$ and $\partial_{m_i}\mathbb{E}_{q(x_i)}[V]$ are the *same* Gaussian-averaged NUFFT (the closed form (47)), computed off-graph and differentiated through the S -envelope by autodiff — so the injected ∂_m is the exact envelope-damped $\mathbb{E}_{q(x_i)}[\nabla V]$, *not* the point value $\nabla V(m_i)$ — and only ∂_S is Hessian-dropped. ω_r (= the $D_\theta A_r^{-1} D_\theta$ diagonal sums) is Hutchinson-estimated; `FrozenEFGPDriftWithVHutch`’s `custom_vjp` injects the value and m -gradient and zeroes the S -slot.

End-to-end: from `trm` gradients to a natural-grad step

1. Pre-compute drift moments $(\text{Ef}, \text{Eff}, \text{Edfdx}) = (\mathbb{E}_{q(x_i)}[\bar{f}], \|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2, \mathbb{E}_{q(x_i)}[J_{\bar{f}}])$ at all transition base points via `drift_moments_gmix_jax`; stored on `FrozenEFGPDrift`.
2. `nat_grad_transition (sing/sing.py)` runs `vmap(grad(compute_neg_CE_single))` over transitions.
3. `compute_neg_CE_single` consumes *mean parameters* $\mu = (\mathbb{E}[x_i], \mathbb{E}[x_i x_i^\top], \mathbb{E}[x_i x_{i+1}^\top])$, internally runs `mean_to_marginal_params` to recover $(m_i, S_i, S_{i,i+1})$, and assembles `trm` from items (i)-(iii).
4. `jax.grad` chain-rules: (i) pure autodiff through traces; (ii) autodiff + `ef_with_jac_grad`’s injected $\partial \text{Ef} / \partial m$; (iii) entirely via `eff_with_grads`’s injected $\partial \text{Eff} / \partial (m, S)$. Returns $\partial \mathcal{L} / \partial \mu$ — which by (31) is the natural gradient on (J, h, L) . SING ρ -damps and applies.

8 Algorithm summary and complexity

One outer iteration of the E-step

Given current $q(x_{0:T})$ (marginals m_i, S_i and cross-covariances $S_{i,i+1}$), kernel hyperparameters θ , diffusion Σ , and spectral grid $\{\xi_k\}$ with weights D_θ :

1. **Increments.** $d_i = m_{i+1} - m_i$, $C_i = S_{i,i+1} - S_i$.
2. **Sources.** `_flatten_stein` packs $\{(m_i, S_i, d_i, C_i, \Delta_i)\}_{k,i}$ into $N = K(T-1)$ flat supersources (padded slots get $S_i=I$ and weight 0). For `'gmix'/'analytic'` the sources are the $q(x_i) = \mathcal{N}(m_i, S_i)$ Gaussians themselves; for `'mc'`, draw $x_i^{(s)} \sim q(x_i)$.
3. **Precision.** Build the BTTB generator T_r from weighted sources by the resolved backend (§6: closed-form spreader / Taylor-envelope NUFFT / plain Type-1 NUFFT) with weights Δ_i/σ^2 ; A_r acts as $v \mapsto v + D_\theta T_r D_\theta v$.

4. **RHS.** Same primitive with weights $d_{i,r}/\sigma^2$ and $[C_i]_{j,r}/\sigma^2$ produces $h_{1,r}$ and $h_{2,r}$.
5. **Solve.** Jacobi-preconditioned CG: $A_r \mu_r = h_r$.
6. **Drift moments.** All three backends first return point-eval moments (`drift_moments_jax`); with the default `qx_moments_method='gmix_batched'` these are then *overwritten* by the Gaussian-averaged $\mathbb{E}_{q(x_i)}[\bar{f}]$, $\mathbb{E}_{q(x_i)}[J\bar{f}]$, $\|\mathbb{E}_{q(x_i)}[\bar{f}]\|^2$ from `gmix-gather` (`drift_moments_gmix_jax`; §7). Setting `'linearised_shim'` keeps the point-evals (App. C).
7. **$q(x)$ update.** Feed these into `compute_neg_CE_single` via `FrozenEFGPDrift`; `jax.grad` inside `nat_grad_transition` returns $\partial\mathcal{L}/\partial\mu$, which by (31) is the natural gradient on (J, h, L) . SING ρ -damps and applies.

Complexity

Let N be the source count ($N = K(T-1)$ for the `gmix` path, $N = K(T-1)S$ for MC; K is the number of trials), M the number of Fourier features, k the CG iteration count. The $q(f)$ update costs

$$\underbrace{O(N)}_{\text{spread / cloud assembly}} + \underbrace{O(M \log M)}_{\text{NUFFT / FFT}} + \underbrace{O(k D_x M \log M)}_{\text{CG over output dims}}. \quad (44)$$

For `'analytic'` the $O(N)$ term becomes $O(P(p)N)$ with $P(p) = \binom{p+2}{2}$, but as one batched transform. Drift moments at T points cost $O(T) + O(D_x M \log M)$. The optional V -restoration of App. B adds, *per outer iter*, $O(P D_x k M \log M)$ for the P Hutchinson probes that build $\hat{\omega}_r$, and *per inner iter* $O((1+D_x)(M \log M + N))$ for the $1+D_x$ batched NUFFTs that produce $(V_i, \nabla V_i)$ at the current $\{m_i\}$ (`efgp.em.py` calls `precompute_V_and_grad_E_V_per_t` inside `one_iter`); only the CG solves are hoisted to the outer iter. The custom-VJP shim itself is a table lookup. The implementation rule is to never materialise K , Φ , $\Phi^* W \Phi$, or A_r^{-1} : every required action is a Fourier matvec, a BTTB matvec, or a CG solve.

A Math-to-code dictionary

Math	Code
Spectral grid weights $D_\theta = \text{diag}(\sqrt{S(\xi_k)} h^d)$	<code>grid.ws</code> (real part)
Frequency points ξ_k	<code>grid.xis_flat</code>
Adjoint Type-1 NUFFT F_X^*	<code>nufft1</code>
Forward Type-2 NUFFT $\phi(x)^* \mu$	<code>nufft2</code>
BTTB generator $T_r = F_X^* W_r F_X$	<code>bttb_conv_vec_weighted</code> $\rightarrow$ <code>make_toeplitz</code>
Operator $A_r v = v + D_\theta T_r D_\theta v$	<code>make_A_apply</code>
Solve $A_r \mu_r = h_r$ (Jacobi-preconditioned CG)	<code>cg_solve</code>
$h_r = h_{1,r} + h_{2,r}$ build, 'mc' backend	<code>compute_mu_r_jax</code>
same, 'gmix' backend (§6.1)	<code>compute_mu_r_gmix_jax + sing/efgp_gmix_spreader.py</code>
same, 'analytic' backend (§6.2)	<code>compute_mu_r_analytic_jax, _env_taylor_pairs</code>
$q(f)$ update + drift moments in one call (per backend)	<code>qf_and_moments_{jax, gmix_jax, analytic_jax}</code>
$\bar{f}(m_i)$, $J_{\bar{f}}(m_i)$ point-eval ('linearised_shim')	<code>drift_moments_jax</code>
$\mathbb{E}_{q(x_i)}[\bar{f}]$, $\mathbb{E}_{q(x_i)}[J_{\bar{f}}]$ Gaussian-averaged (default)	<code>drift_moments_gmix_jax</code> + <code>sing/efgp_gmix_gather.py</code>
$\mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}]$ exact closed form (<i>defined, unused in the fit</i>)	<code>gmix.E_full_Eff</code> (<code>sing/efgp_gmix_qx_moments.py</code>)
Spectral autocorrelation $\rho(\delta)$ precompute (one FFT/outer iter)	<code>precompute_aux</code> (only feeds the V-Hutch path)
Multi-trial flatten of $\{(m_i, S_i, d_i, C_i, \Delta_i)\}_{k,i}$	<code>_flatten_stein</code>
$\text{diag}(\Phi A_r^{-1} \Phi^*)$ (general Hutchinson primitive; tests only)	<code>hutchinson_diag</code>
$\omega_r(\delta)$ Hutchinson estimator (eq. 48)	<code>precompute_omega_per_r</code> (<code>sing/efgp_qx_v_hutch.py</code>)
$(V_i, \nabla V_i)$ batched NUFFT-2 precompute (eq. 47)	<code>precompute_V_and_grad_E_V_per_t</code>
V-Hutch custom-VJP shim into per-transition $q(x)$ update	<code>FrozenEFGPDriftWithVHutch</code> (<code>efgp_qx_v_hutch.py:485</code>)
GN custom-VJP shim into $q(x)$ update (default, §2)	<code>FrozenEFGPDrift, ef_with_jac_grad, eff_with_grads</code>
$q(x)$ consumer: <code>vmap(grad(trm))</code> $\rightarrow (J, h, L)$	<code>nat_grad_transition</code> (<code>sing/sing.py:46</code>)
<code>trm</code> assembly from $(\text{Ef}, \text{Eff}, \text{Edfdx})$	<code>compute_neg_CE_single</code> (<code>sing_helpers.py:289</code>)
EM driver, backend resolution, M-step	<code>fit_efgp_sing_jax</code> (<code>efgp_em.py:404</code>)

$q(x)$ statistic	Code
m_i (marginal mean)	<code>ms</code>
S_i (marginal covariance)	<code>Ss</code>
$\text{Cov}(x_i, x_{i+1}) = S_{i,i+1}$	<code>SSs.transpose(0, 2, 1)</code>
$d_i = m_{i+1} - m_i$	<code>d_a</code>
$C_i = S_{i,i+1} - S_i$	<code>C_a</code>

B V-Hutch restoration (opt-in)

Default production drops the V term entirely (§7). For problem classes where the geometric argument is suspect — boundary or cluster-supported coverage, anisotropic coverage, sharp turns near $\partial\Omega$, cold-start outer iters before $q(x)$ has localised — this appendix gives a Hutchinson + NUFFT restoration whose (m_i, S_i) gradient structure is exactly the linear case of §7. Wiring guard (`efgp_em.py:151--153`): the restoration is active only for `qx_moments_method` $\in$ ('linearised_shim', 'gmix_batched') and `estep_method` $\in$ ('gmix', 'analytic'); with 'mc' it is silently inactive.

B.1 Closed-form $\mathbb{E}_{q(x_i)}[V]$ via BTTB-diagonal sums of $DA^{-1}D$

V_r is a quadratic form in the Fourier features, so — exactly like the mean-square $\bar{f}_r^2 = \phi^* \mu_r \mu_r^* \phi$ of item (iii) — it is a sum of difference-frequency modes,

$$V_r(x) = \sigma_r^{-2} \phi(x)^* A_r^{-1} \phi(x) = \sigma_r^{-2} \sum_{k,\ell} D_k (A_r^{-1})_{k\ell} D_\ell e^{2\pi i(\xi_\ell - \xi_k)^\top x}. \quad (45)$$

The dense A_r^{-1} (unlike the rank-one $\mu_r \mu_r^*$) only sets the mode coefficients; the x -dependence is still $e^{2\pi i \delta^\top x}$, $\delta := \xi_\ell - \xi_k$, so each mode averages under $q(x_i)$ by the same characteristic function (23). Collecting modes by lag, define the diagonal sums

$$\omega_r(\delta) := \sum_{(k,\ell): \xi_\ell - \xi_k = \delta} D_k (A_r^{-1})_{k\ell} D_\ell, \quad (46)$$

the BTTB projection of $D_\theta A_r^{-1} D_\theta$ onto lags δ on the $2M$ -padded spectral-difference grid. By stationarity of the kernel, $\omega_r(-\delta) = \omega_r(\delta)^*$ (Hermitian-symmetric; generally complex when source locations break centrosymmetry). Then

$$\mathbb{E}_{q(x_i)}[V_r(x_i)] = \sum_\delta \omega_r(\delta) e^{2\pi i \delta^\top m_i} e^{-2\pi^2 \delta^\top S_i \delta}, \quad (47)$$

a Gaussian-enveloped Type-2 NUFFT against the difference-grid coefficient block — the primitive `gmix_E_full_Eff` already uses for $\mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}]$, with ω_r replacing the autocorrelation ρ_r of μ_r .

So the recipe is modular: (i) obtain the drift posterior variance — i.e. the coefficients $\omega_r(\delta)$ — by *any* estimator (the Hutchinson estimator below, or a pathwise/Matheron posterior sampler, Wilson et al. 2020); (ii) apply the gmix envelope-NUFFT (47). Only step (i) touches A_r^{-1} ; step (ii), the $q(x_i)$ -average, is identical to every other moment.

B.2 Hutchinson estimator of ω_r

ω_r is estimated unbiasedly by a padded-FFT pair on P Rademacher probes $z_p \in \{\pm 1\}^M$:

$$\hat{\omega}_r = \frac{1}{P} \sum_{p=1}^P \mathcal{F}^{-1}(\overline{\mathcal{F}[Du_{p,r}]} \cdot \mathcal{F}[Dz_p]), \quad u_{p,r} = A_r^{-1} z_p, \quad (48)$$

with $A_r^{-1} z_p$ via the same Jacobi-preconditioned CG that drives the $q(f)$ update. Variance scales as $\|DA^{-1}D\|_F/\sqrt{P}$; $P=4$ probes shared across the D_{out} output dims are sufficient. *Keep the imaginary part:* a `.real` cast on the IFFT discards $-\Im \omega(\delta) \sin(2\pi \delta^\top m)$, the dominant error vs dense ground truth when sources are not centrosymmetric.

B.3 Per-transition gradient: exact in m , S -slot dropped

Following (43), the shim keeps the value and m -gradient exact and drops only the S -slot:

$$\mathbb{E}_{q(x_i)}[V(x_i)] \text{ (exact)}, \quad \partial_{m_i} \mathbb{E}[V] = \mathbb{E}[\nabla V] \text{ (exact)}, \quad \partial_{S_i} \mathbb{E}[V] = 0. \quad (49)$$

`precompute_V_and_grad_E_V_per_t` evaluates the closed form (47) at $\{m_i\}$ and takes `jax.grad` w.r.t. $\{m_i\}$, so ∂_m is the exact envelope-damped $\mathbb{E}[\nabla V]$ (not $\nabla V(m_i)$); `FrozenEFGPDriftWithVHutch` zeroes the $\partial V_i / \partial S_i$ slot. This is the V -analog of the item-(ii) $\bar{f}$ shim, at the same truncation (drop ∂_S only).

The value V_i and the m -gradient are exact (the closed form (47) and its autodiff); the *only* dropped term is the S -slot $\partial_{S_i} \mathbb{E}[V] = \frac{1}{2} \mathbb{E}_{q(x_i)}[\nabla^2 V]$, of size $O(\|S_i\| \cdot \sigma_f^2 / \ell^2)$ for SE kernels — small for $q(x)$ localised at scale $\sim \ell$. To restore this S -pull exactly, replace V by its second-order Taylor with PSD-projected Hessian $H_V^+ \succeq 0$ (Price’s identity, $\partial_S = \frac{1}{2} H_V^+$); cost: $D(D+1)/2$ extra batched NUFFT’s against frequency-modulated $\omega(\delta) \cdot (-2\pi^2 \delta_j \delta_k) \text{env}(\delta)$ coefficient grids. Not enabled by default.

B.4 Homogeneous- S vs heterogeneous- S sub-modes

Two production sub-modes for the per-source envelope in (47):

‘hutch’ (default *when restoration is on*; ‘none’ is the production default). S_i is replaced by the trajectory-mean $\bar{S}$ in the envelope $e^{-2\pi^2 \delta^\top \bar{S} \delta}$, so the envelope folds into the coefficient grid and the per-source NUFFT-2 is a standard (non-gmix) transform. $\hat{\omega}_r$ is built once per *outer* iter; $(V_i, \nabla V_i)$ are rebuilt each *inner* iter at the current $\{m_i\}$ (and $\bar{S}$ from the current S ’s).

‘hutch_hetS’. Per-source S_i envelope, implemented via the Type-2 gmix-gather primitive (`efgp.gmix.gather.py`). Pipeline: (i) inverse-FFT $\hat{\omega}(\delta)$ once to a uniform spatial grid $\tilde{\omega}(x)$; (ii) for each source i , gather $\tilde{\omega}$ via a per-source Gaussian stencil weighted by $\mathcal{N}(x; m_i, S_i)$ and Riemann-summed over a $(2r+1)^D$ neighbourhood, with $r = \lceil n_\sigma \sigma_{\text{max}} / h_{\text{grid}} \rceil$ (static, picked at startup from V_0). Both V_i and ∇V_i flow through the same primitive (gradient via `jax.grad` reverse-mode through the gather), rebuilt every inner iter.

C Implementation and benchmarks

C.1 Custom-VJP shims: why a shim, not pure autodiff

`nat_grad_transition` runs `vmap(grad(loss_one_transition))` where `loss_one_transition` closes over a single (m_t, S_t) . Inside that `vmap`, JAX/XLA does *not* hoist the FFT across `vmap`'d `nufft2` calls: a per-source NUFFT-2 inside `ff(t, m, S)` pays the full $O(M \log M)$ FFT cost *per source*, giving $O(NM \log M)$ per inner iter and $\sim 50\times$ slowdown at $K=10/T=500$ (measured). `FrozenEFGPDrift` sidesteps this by precomputing $(\bar{f}, J_{\bar{f}})$ at all sources via ONE forward batched NUFFT-2 plus its AD-driven adjoint (D Type-1 NUFFTs) *outside* the per-transition `vmap`, then injecting both as constants per-transition via `custom_vjp` with the §7 gradient table. The V-Hutch shim `FrozenEFGPDriftWithVHutch` follows the same pattern for $(V_i, \nabla V_i)$. A refactor of `nat_grad_transition` to compute the multi-transition ELBO in one shot and `jax.grad` it once over all (m_i, S_i) would recover the same asymptotic cost without shims.

C.2 Point-eval alternative: linearised-shim

The §7 default Gaussian-averages $(\bar{f}, J_{\bar{f}})$ under $q(x_i)$ via the `gmix-gather` primitive. A cheaper alternative, *linearised-shim*, uses point-eval $(f(m_i), J_{\bar{f}}(m_i))$ directly (no Gaussian averaging): one batched Type-2 NUFFT on μ_r at the $\{m_i\}$ source set, with no gather. The Taylor-truncated gradient structure of §7 is identical — so both consume the same `FrozenEFGPDrift` custom-VJP — but the base values miss the first-order S_i -correction. Empirically (§C wall-time table) the two paths reach the same fixed point to 3–4 decimals and within $\sim 7\%$ wall, so `gmix-batched` is the production default but *linearised-shim* is retained for problems where the gather truncation is not negligible (small ℓ regime) or for debugging.

C.3 Wall-time comparisons ($K = 10, T = 500$)

Benchmarks against the '`gmix_batched`' baseline, subprocess-isolated timings, warm JIT:

- Base-point choice: '`gmix_batched`' reaches the same $(\hat{\ell}, \hat{\sigma}_f^2, \text{lat_rmse})$ fixed point as '`linearised_shim`' to 3–4 decimals, at wall within $\sim 7\%$ (`demos/bench.qx_moments_modes.py`).
- V-restoration overhead: '`hutch`' ~ 53.7 s warm vs '`none`' ~ 48.3 s ($\sim 11\%$); '`hutch_hetS`' ~ 64.8 s ($\sim 1.34\times$). Recovery is bit-identical across all three modes (`demos/bench_hutch_vs_none.py`). *Caveat:* the `efgp_em.py` docstring quotes $\sim 22\%$ for '`hutch_hetS`' at the same K, T — the two figures come from different runs; re-measure before quoting either in the manuscript.

C.4 ∇V magnitude diagnostic

Setup: $K=10$ trials, $x_0 \sim \mathcal{U}([-2, 2]^2)$, integrated for 8 s with $\sigma=0.4$, $\ell_{\text{true}}=0.6$. Latent paths cover $\Omega \approx [-3.5, 3]^2$ with $\sim 5,000$ sources, $|\Omega|/\ell^d \approx 120$; the §7-A isotropic-coverage argument applies with margin.

Direct measurement at a representative transition shows the wiring is correct (the `custom_vjp` bwd produces ∇V matching the precomputed `grad_V_const` to numerical precision) but the *magnitude* of ∇V is $\sim 5 \times 10^{-5}$ of the $\partial \text{Eff}/\partial m = 2J^\top \mathbb{E}[\bar{f}]$ term per transition, translating to $\sim 0.16\%$ change in the full `compute_neg_CE_single` gradient direction. SING's ρ -damping washes this out. The bit-identical recovery between '`none`' and '`hutch`' (final $\hat{\ell}=1.1337$, $\hat{\sigma}_f^2=1.3230$, latent RMSE=0.0646, all matching to 3–4 decimals across all three V-restoration modes) reflects the structural argument, not a wiring bug.

C.5 Op count for V-Hutch per outer iter

Operation	Count	Per-call cost
Hutch CG ($A_r^{-1} z_p$, eq. 48)	$P \cdot D_{\text{out}}$	$O(k_{\text{cg}} M \log M)$
FFT cross-correlation (gives $\hat{\omega}_r$)	$P \cdot D_{\text{out}}$	$O(M \log M)$
Forward batched NUFFT-2 (gives $\{V_i\}$) — <i>per inner iter</i>	1	$O(K(M \log M + T))$
Adjoint NUFFT-1 (∇V_i via <code>jax.grad</code>) — <i>per inner iter</i>	D	$O(K(M \log M + T))$
Per-inner-iter CG solves added		0

NUFFTs are the only ops that scale with $N = K(T-1)$. Only the $\hat{\omega}_r$ block (CG + FFT cross-correlation) is hoisted to the outer iter; the $1+D$ batched NUFFTs that produce $(V_i, \nabla V_i)$ sit *inside one_iter*, since they must be re-evaluated at the current $\{m_i\}$, so they scale with n_{estep} . At $K=10/T=500$ ($D=2$, $M \sim 10^3$, $P=4$, $k_{\text{cg}} \sim 30$): CG block ~ 0.1 s per outer iter, NUFFT block ~ 1 ms per inner iter — consistent with the $\sim 11\%$ measured wall for '`hutch`'.